Wine Quality Prediction Using Statistical Learning

Madison Montgomery

2025-12-05

1. Introduction

Wine quality is usually assessed by expert tasters, but we're interested in whether chemical properties alone can predict quality ratings. This project uses the Wine Quality Dataset from the UCI Machine Learning Repository to build predictive models.

The dataset contains 6,497 wine samples (1,599 red and 4,898 white) with 11 chemical measurements and a quality score from 0-10. I'll be applying various statistical learning methods including linear regression, regularization techniques, classification models, and dimensionality reduction to see which approaches work best and which chemical properties matter most for wine quality.

2. Data Loading and Preparation

```r
set.seed(123)
options(width = 120)

# Load data from UCI repository
url_red  <- "https://archive.ics.uci.edu/ml/machine-learning-databases/wine-quality/winequality-red.csv
url_white <- "https://archive.ics.uci.edu/ml/machine-learning-databases/wine-quality/winequality-white.

red  <- read.csv(url_red, sep = ";")
red$type <- "red"

white <- read.csv(url_white, sep = ";")
white$type <- "white"

# Combine datasets
wine_all <- rbind(red, white)
wine_all$type <- as.factor(wine_all$type)

cat("Dataset successfully loaded.\n")
```

```
## Dataset successfully loaded.
```

```r
cat("Total observations:", nrow(wine_all), "\n")
```

```
## Total observations: 6497
```

```r
cat("Red wines:", nrow(red), "| White wines:", nrow(white), "\n")
```

```
## Red wines: 1599 | White wines: 4898
```

The data loaded without any issues and there are no missing values, so we can move straight into exploring it.

3. Exploratory Data Analysis

3.1 Summary Statistics

```r
summary(wine_all)
```

```
##  fixed.acidity    volatile.acidity  citric.acid     residual.sugar     chlorides       free.sulfur.di
##  Min.   : 3.800   Min.   :0.0800   Min.   :0.0000   Min.   : 0.600   Min.   :0.00900   Min.   :  1.0
##  1st Qu.: 6.400   1st Qu.:0.2300   1st Qu.:0.2500   1st Qu.: 1.800   1st Qu.:0.03800   1st Qu.: 17.0
##  Median : 7.000   Median :0.2900   Median :0.3100   Median : 3.000   Median :0.04700   Median : 29.0
##  Mean   : 7.215   Mean   :0.3397   Mean   :0.3186   Mean   : 5.443   Mean   :0.05603   Mean   : 30.5
##  3rd Qu.: 7.700   3rd Qu.:0.4000   3rd Qu.:0.3900   3rd Qu.: 8.100   3rd Qu.:0.06500   3rd Qu.: 41.0
##  Max.   :15.900   Max.   :1.5800   Max.   :1.6600   Max.   :65.800   Max.   :0.61100   Max.   :289.0
##  total.sulfur.dioxide    density             pH          sulphates         alcohol         quality
##  Min.   :  6.0        Min.   :0.9871   Min.   :2.720   Min.   :0.2200   Min.   : 8.00   Min.   :3.00
##  1st Qu.: 77.0        1st Qu.:0.9923   1st Qu.:3.110   1st Qu.:0.4300   1st Qu.: 9.50   1st Qu.:5.00
##  Median :118.0        Median :0.9949   Median :3.210   Median :0.5100   Median :10.30   Median :6.00
##  Mean   :115.7        Mean   :0.9947   Mean   :3.219   Mean   :0.5313   Mean   :10.49   Mean   :5.81
##  3rd Qu.:156.0        3rd Qu.:0.9970   3rd Qu.:3.320   3rd Qu.:0.6000   3rd Qu.:11.30   3rd Qu.:6.00
##  Max.   :440.0        Max.   :1.0390   Max.   :4.010   Max.   :2.0000   Max.   :14.90   Max.   :9.00
```

```r
cat("\nNumber of observations:", nrow(wine_all), "\n")
```

```
##
## Number of observations: 6497
```

```r
cat("Number of variables:", ncol(wine_all), "\n")
```

```
## Number of variables: 13
```

Looking at the summary statistics, most wines are rated around 5-6 (median = 6). The quality scores range from 3 to 9, so we don't see any perfect 10s or terrible 0s in this dataset. Alcohol content varies quite a bit from 8% to almost 15%, and residual sugar has a huge range which probably reflects different wine styles from bone dry to sweet. Some variables like citric acid and chlorides look pretty skewed based on the min/max values.

3.2 Correlation Analysis

```r
# Examine correlations among predictors
num_vars <- wine_all[, sapply(wine_all, is.numeric)]
num_vars <- num_vars[, !(names(num_vars) %in% "quality")]
cor_mat <- cor(num_vars)
print(round(cor_mat, 2))
```

```
##                      fixed.acidity volatile.acidity citric.acid residual.sugar chlorides free.sulfur
## fixed.acidity                 1.00             0.22        0.32          -0.11      0.30
## volatile.acidity              0.22             1.00       -0.38          -0.20      0.38
## citric.acid                   0.32            -0.38        1.00           0.14      0.04
## residual.sugar               -0.11            -0.20        0.14           1.00     -0.13
## chlorides                     0.30             0.38        0.04          -0.13      1.00
## free.sulfur.dioxide          -0.28            -0.35        0.13           0.40     -0.20
## total.sulfur.dioxide         -0.33            -0.41        0.20           0.50     -0.28
## density                       0.46             0.27        0.10           0.55      0.36
## pH                           -0.25             0.26       -0.33          -0.27      0.04
## sulphates                     0.30             0.23        0.06          -0.19      0.40
## alcohol                      -0.10            -0.04       -0.01          -0.36     -0.26
##                      total.sulfur.dioxide density    pH sulphates alcohol
## fixed.acidity                       -0.33    0.46 -0.25      0.30   -0.10
## volatile.acidity                    -0.41    0.27  0.26      0.23   -0.04
## citric.acid                          0.20    0.10 -0.33      0.06   -0.01
## residual.sugar                       0.50    0.55 -0.27     -0.19   -0.36
## chlorides                           -0.28    0.36  0.04      0.40   -0.26
## free.sulfur.dioxide                  0.72    0.03 -0.15     -0.19   -0.18
## total.sulfur.dioxide                 1.00    0.03 -0.24     -0.28   -0.27
## density                              0.03    1.00  0.01      0.26   -0.69
## pH                                  -0.24    0.01  1.00      0.19    0.12
## sulphates                           -0.28    0.26  0.19      1.00    0.00
## alcohol                             -0.27   -0.69  0.12      0.00    1.00
```

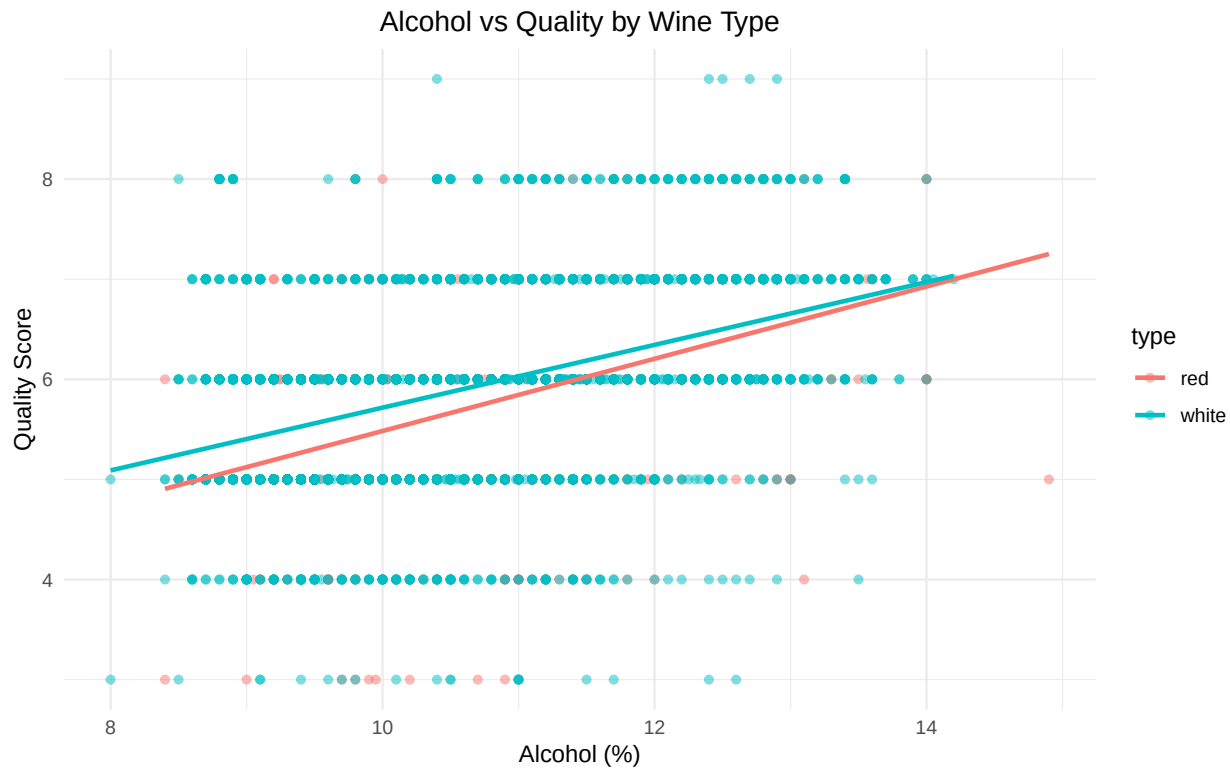
There are some notable correlations between certain predictors. Alcohol and density have a strong negative correlation of -0.69, which makes sense since alcohol is less dense than water. Fixed acidity and citric acid show a moderate correlation of 0.32, and fixed acidity and pH are negatively correlated at -0.25 (lower pH means more acidic). Density and residual sugar have a correlation of 0.55, which is expected since sugar affects how dense the wine is. Free and total sulfur dioxide are correlated at 0.72 since free SO2 is part of the total.

These correlations suggest multicollinearity might be an issue for regular linear regression, which is why regularization methods like Ridge and Lasso could potentially perform better.

3.3 Visualizations

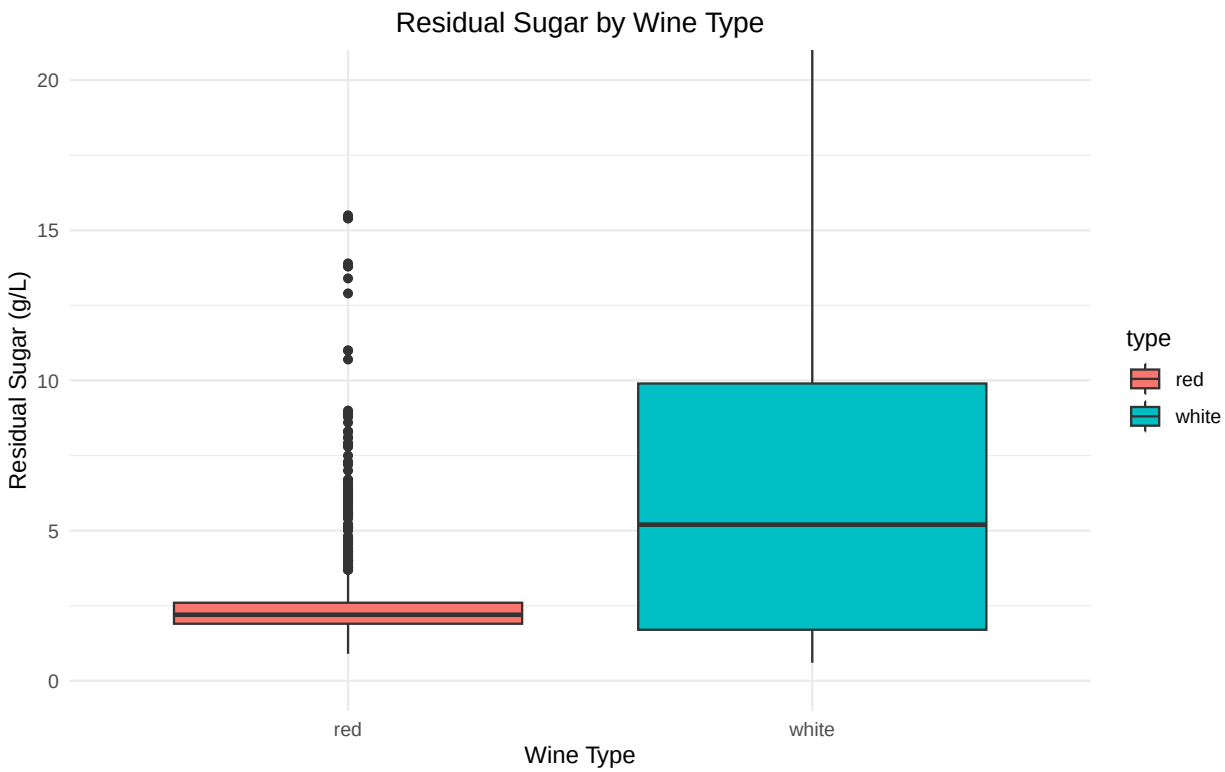
```r
library(ggplot2)

# Alcohol vs Quality by Wine Type
ggplot(wine_all, aes(x = alcohol, y = quality, color = type)) +
  geom_point(alpha = 0.5) +
  geom_smooth(method = "lm", se = FALSE) +
  theme_minimal() +
  labs(title = "Alcohol vs Quality by Wine Type",
```

```
       x = "Alcohol (%)", y = "Quality Score") +
  theme(plot.title = element_text(hjust = 0.5))
```



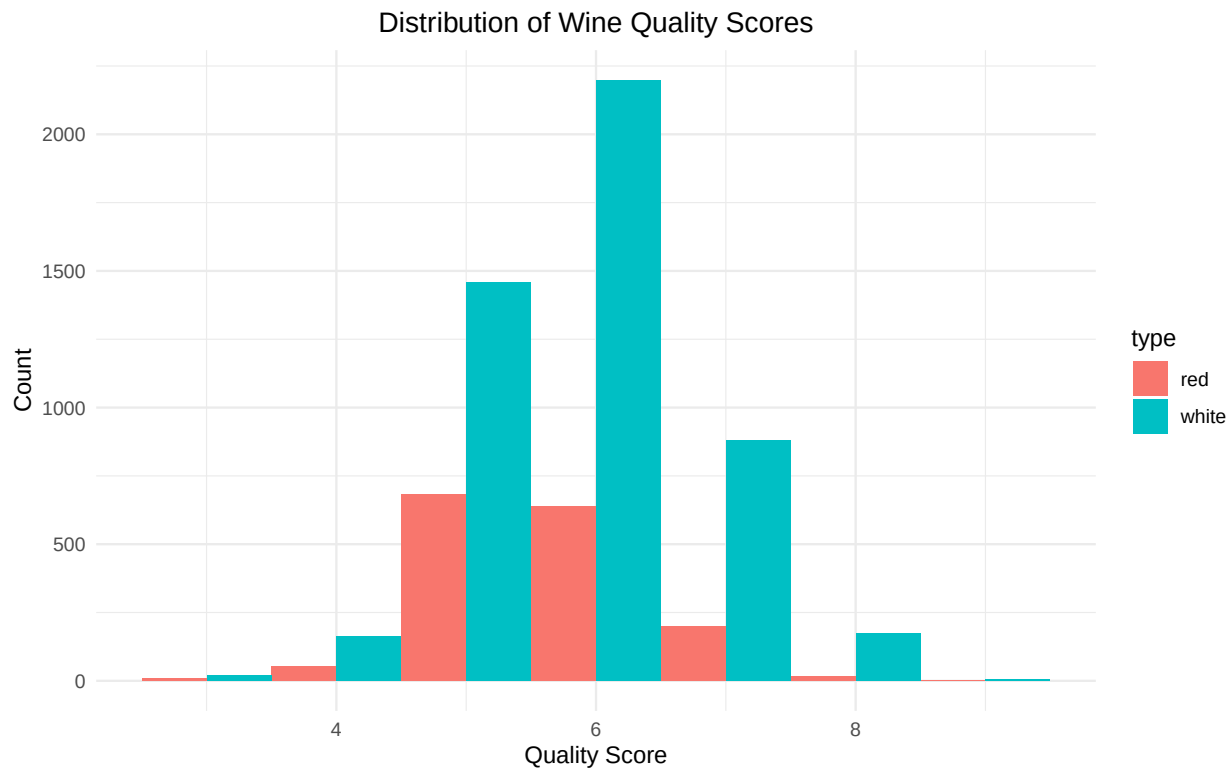
Higher alcohol content seems to be associated with higher quality ratings for both wine types. The relationship looks pretty linear, though there's a lot of scatter. Red and white wines follow similar patterns here.

```
# Residual Sugar by Type
ggplot(wine_all, aes(x = type, y = residual.sugar, fill = type)) +
  geom_boxplot() +
  theme_minimal() +
  labs(title = "Residual Sugar by Wine Type",
       x = "Wine Type", y = "Residual Sugar (g/L)") +
  theme(plot.title = element_text(hjust = 0.5)) +
  coord_cartesian(ylim = c(0, 20))
```



White wines have noticeably higher residual sugar levels than red wines on average. This makes sense since white wines include more semi-sweet and sweet varieties, while reds are typically drier.

```
# Distribution of Quality Scores
ggplot(wine_all, aes(x = quality, fill = type)) +
  geom_histogram(binwidth = 1, position = "dodge") +
  theme_minimal() +
  labs(title = "Distribution of Wine Quality Scores",
       x = "Quality Score", y = "Count") +
  theme(plot.title = element_text(hjust = 0.5))
```



Quality scores look approximately normal and centered around 5-6. Very few wines get extreme ratings (either really good or really bad), which suggests the raters tend to give moderate scores. The distributions are pretty similar for red and white wines.

4. Regression Modeling

Now I'll predict quality as a continuous numeric outcome using different regression methods.

4.1 Model Preparation

```
library(caret)

predictors <- c("fixed.acidity", "volatile.acidity", "citric.acid", "residual.sugar",
                "chlorides", "free.sulfur.dioxide", "total.sulfur.dioxide", "density",
                "pH", "sulphates", "alcohol")

df_model <- wine_all[, c(predictors, "quality", "type")]

# Set up 10-fold cross-validation
train_ctrl <- trainControl(method = "cv", number = 10)
```

4.2 Multiple Linear Regression

```r
ols_fit <- train(quality ~ ., data = df_model[, !(names(df_model) %in% "type")],
                 method = "lm", trControl = train_ctrl)
print(ols_fit)
```

```
## Linear Regression
##
## 6497 samples
##   11 predictor
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 5848, 5846, 5847, 5848, 5848, 5848, ...
## Resampling results:
##
##   RMSE       Rsquared   MAE
##   0.7359456  0.2905281  0.5693415
##
## Tuning parameter 'intercept' was held constant at a value of TRUE
```

```r
summary(ols_fit$finalModel)
```

```
##
## Call:
## lm(formula = .outcome ~ ., data = dat)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -3.7569 -0.4597 -0.0412  0.4694  2.9907
##
## Coefficients:
##                        Estimate Std. Error t value Pr(>|t|)
## (Intercept)           5.576e+01  1.189e+01   4.688 2.81e-06 ***
## fixed.acidity         6.768e-02  1.557e-02   4.346 1.41e-05 ***
## volatile.acidity     -1.328e+00  7.737e-02 -17.162  < 2e-16 ***
## citric.acid          -1.097e-01  7.962e-02  -1.377    0.168
## residual.sugar        4.356e-02  5.156e-03   8.449  < 2e-16 ***
## chlorides            -4.837e-01  3.327e-01  -1.454    0.146
## free.sulfur.dioxide   5.970e-03  7.511e-04   7.948 2.22e-15 ***
## total.sulfur.dioxide -2.481e-03  2.767e-04  -8.969  < 2e-16 ***
## density              -5.497e+01  1.214e+01  -4.529 6.04e-06 ***
## pH                    4.393e-01  9.037e-02   4.861 1.20e-06 ***
## sulphates             7.683e-01  7.612e-02  10.092  < 2e-16 ***
## alcohol               2.670e-01  1.673e-02  15.963  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.7353 on 6485 degrees of freedom
## Multiple R-squared:  0.2921, Adjusted R-squared:  0.2909
## F-statistic: 243.3 on 11 and 6485 DF,  p-value: < 2.2e-16
```

The basic linear regression model gets a cross-validated RMSE of about 0.73 and an R-squared around 0.29. So we're explaining roughly 29% of the variation in quality scores. The significant predictors include alcohol (positive effect), volatile acidity (negative), and sulphates (positive).

An R-squared of 0.29 is pretty modest though, which suggests that chemical properties alone don't tell the whole story. Quality ratings probably depend on subjective factors like taste preferences that aren't captured in these measurements.

4.3 Subset Selection

I'll use forward, backward, and best subset selection to see if we can get similar performance with fewer predictors.

```r
library(leaps)

# Custom function for cross-validated subset selection
cv_subset <- function(formula, data, k = 10) {
  folds <- createFolds(data$quality, k = k)
  rmse_vec <- numeric(k)
  r2_vec <- numeric(k)

  for (i in 1:k) {
    train_idx <- unlist(folds[-i])
    test_idx  <- unlist(folds[i])
    train_data <- data[train_idx, ]
    test_data  <- data[test_idx, ]

    fit <- lm(formula, data = train_data)
    preds <- predict(fit, newdata = test_data)

    rmse_vec[i] <- sqrt(mean((preds - test_data$quality)^2))
    r2_vec[i]   <- 1 - sum((preds - test_data$quality)^2) /
                       sum((mean(train_data$quality) - test_data$quality)^2)
  }

  list(RMSE = mean(rmse_vec), Rsquared = mean(r2_vec))
}

# Forward Stepwise Selection
forward_fit <- regsubsets(quality ~ .,
                          data = df_model[, !(names(df_model) %in% "type")],
                          nvmax = length(predictors), method = "forward")
forward_sum <- summary(forward_fit)
best_forward_vars <- names(coef(forward_fit, which.min(forward_sum$bic)))[-1]
forward_formula <- as.formula(paste("quality ~", paste(best_forward_vars, collapse = "+")))

cat("\nForward Stepwise - Selected Variables:\n")
```

```
##
## Forward Stepwise - Selected Variables:
```

```r
print(best_forward_vars)
```

```
##  [1] "fixed.acidity"        "volatile.acidity"     "residual.sugar"       "chlorides"            "fre
##  [6] "total.sulfur.dioxide" "density"              "pH"                   "sulphates"            "alc
```

```r
forward_cv <- cv_subset(forward_formula, df_model[, !(names(df_model) %in% "type")])
cat("\nForward Stepwise CV Metrics:\n")
```

```
##
## Forward Stepwise CV Metrics:
```

```r
print(forward_cv)
```

```
## $RMSE
## [1] 0.7361308
##
## $Rsquared
## [1] 0.2889713
```

```r
# Backward Stepwise Selection
backward_fit <- regsubsets(quality ~ .,
                           data = df_model[, !(names(df_model) %in% "type")],
                           nvmax = length(predictors), method = "backward")
backward_sum <- summary(backward_fit)
best_backward_vars <- names(coef(backward_fit, which.min(backward_sum$bic)))[-1]
backward_formula <- as.formula(paste("quality ~", paste(best_backward_vars, collapse = "+")))

cat("\nBackward Stepwise - Selected Variables:\n")
```

```
##
## Backward Stepwise - Selected Variables:
```

```r
print(best_backward_vars)
```

```
## [1] "fixed.acidity"        "volatile.acidity"     "residual.sugar"       "free.sulfur.dioxide"  "tota
## [6] "density"              "pH"                   "sulphates"            "alcohol"
```

```r
backward_cv <- cv_subset(backward_formula, df_model[, !(names(df_model) %in% "type")])
cat("\nBackward Stepwise CV Metrics:\n")
```

```
##
## Backward Stepwise CV Metrics:
```

```r
print(backward_cv)
```

```
## $RMSE
## [1] 0.7361119
##
## $Rsquared
## [1] 0.2891814
```

```r
# Best Subset Selection
best_fit <- regsubsets(quality ~ .,
                       data = df_model[, !(names(df_model) %in% "type")],
                       nvmax = length(predictors))
best_sum <- summary(best_fit)
best_vars <- names(coef(best_fit, which.min(best_sum$bic)))[-1]
best_formula <- as.formula(paste("quality ~", paste(best_vars, collapse = "+")))

cat("\nBest Subset - Selected Variables:\n")
```

```
##
## Best Subset - Selected Variables:
```

```r
print(best_vars)
```

```
## [1] "fixed.acidity"        "volatile.acidity"     "residual.sugar"       "free.sulfur.dioxide"  "tota
## [6] "density"              "pH"                   "sulphates"            "alcohol"
```

```r
best_cv <- cv_subset(best_formula, df_model[, !(names(df_model) %in% "type")])
cat("\nBest Subset CV Metrics:\n")
```

```
##
## Best Subset CV Metrics:
```

```r
print(best_cv)
```

```
## $RMSE
## [1] 0.7356425
##
## $Rsquared
## [1] 0.2893253
```

All three subset selection methods picked pretty similar variables as most important: volatile acidity, free sulfur dioxide, density, pH, sulphates, and alcohol. They're able to simplify the model a bit without really hurting performance, which is nice for interpretation.

4.4 Ridge and Lasso Regression

```r
library(glmnet)

# Ridge Regression (alpha = 0)
ridge_fit <- train(quality ~ .,
                   data = df_model[, !(names(df_model) %in% "type")],
                   method = "glmnet",
                   trControl = train_ctrl,
                   tuneGrid = expand.grid(alpha = 0,
                                          lambda = 10^seq(-3, 1, length.out = 50)))
print(ridge_fit)
```

```
## glmnet
##
## 6497 samples
##   11 predictor
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 5847, 5848, 5847, 5846, 5847, 5848, ...
## Resampling results across tuning parameters:
##
##   lambda        RMSE       Rsquared   MAE
##    0.001000000  0.7364866  0.2898367  0.5709367
##    0.001206793  0.7364866  0.2898367  0.5709367
##    0.001456348  0.7364866  0.2898367  0.5709367
##    0.001757511  0.7364866  0.2898367  0.5709367
##    0.002120951  0.7364866  0.2898367  0.5709367
##    0.002559548  0.7364866  0.2898367  0.5709367
##    0.003088844  0.7364866  0.2898367  0.5709367
##    0.003727594  0.7364866  0.2898367  0.5709367
##    0.004498433  0.7364866  0.2898367  0.5709367
##    0.005428675  0.7364866  0.2898367  0.5709367
##    0.006551286  0.7364866  0.2898367  0.5709367
##    0.007906043  0.7364866  0.2898367  0.5709367
##    0.009540955  0.7364866  0.2898367  0.5709367
##    0.011513954  0.7364866  0.2898367  0.5709367
##    0.013894955  0.7364866  0.2898367  0.5709367
##    0.016768329  0.7364866  0.2898367  0.5709367
##    0.020235896  0.7364866  0.2898367  0.5709367
##    0.024420531  0.7364866  0.2898367  0.5709367
##    0.029470517  0.7364866  0.2898367  0.5709367
##    0.035564803  0.7364866  0.2898367  0.5709367
##    0.042919343  0.7365929  0.2897445  0.5711301
##    0.051794747  0.7368320  0.2895380  0.5715475
##    0.062505519  0.7371573  0.2892659  0.5720590
##    0.075431201  0.7375954  0.2889086  0.5726986
##    0.091029818  0.7381794  0.2884412  0.5734833
##    0.109854114  0.7389485  0.2878336  0.5744406
##    0.132571137  0.7399473  0.2870502  0.5755688
##    0.159985872  0.7412223  0.2860560  0.5769132
##    0.193069773  0.7428295  0.2848054  0.5785343
##    0.232995181  0.7448174  0.2832606  0.5804840
##    0.281176870  0.7472315  0.2813876  0.5827201
##    0.339322177  0.7501121  0.2791551  0.5852233
##    0.409491506  0.7534811  0.2765583  0.5880374
##    0.494171336  0.7573514  0.2736034  0.5911413
##    0.596362332  0.7617214  0.2703148  0.5944873
##    0.719685673  0.7665733  0.2667280  0.5980126
##    0.868511374  0.7718709  0.2629044  0.6016841
##    1.048113134  0.7775645  0.2589186  0.6055110
##    1.264855217  0.7835900  0.2548511  0.6094558
##    1.526417967  0.7898720  0.2507870  0.6136008
##    1.842069969  0.7963207  0.2467987  0.6178753
##    2.222996483  0.8028413  0.2429599  0.6222595
##    2.682695795  0.8093350  0.2393282  0.6268575
```

```
##    3.237457543  0.8157031  0.2359464  0.6316349
##    3.906939937  0.8218529  0.2328430  0.6366505
##    4.714866363  0.8277024  0.2300324  0.6419512
##    5.689866029  0.8331851  0.2275171  0.6473921
##    6.866488450  0.8382521  0.2252903  0.6525980
##    8.286427729  0.8428731  0.2233369  0.6573026
##   10.000000000  0.8470359  0.2216377  0.6614613
##
## Tuning parameter 'alpha' was held constant at a value of 0
## RMSE was used to select the optimal model using the smallest value.
## The final values used for the model were alpha = 0 and lambda = 0.0355648.
```

```r
cat("\nBest Ridge Lambda:", ridge_fit$bestTune$lambda, "\n")
```

```
##
## Best Ridge Lambda: 0.0355648
```

```r
# Lasso Regression (alpha = 1)
lasso_fit <- train(quality ~ .,
                   data = df_model[, !(names(df_model) %in% "type")],
                   method = "glmnet",
                   trControl = train_ctrl,
                   tuneGrid = expand.grid(alpha = 1,
                                          lambda = 10^seq(-4, 0.5, length.out = 50)))
print(lasso_fit)
```

```
## glmnet
##
## 6497 samples
##   11 predictor
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 5848, 5846, 5847, 5847, 5848, 5848, ...
## Resampling results across tuning parameters:
##
##   lambda        RMSE       Rsquared   MAE
##   0.0001000000  0.7359750  0.2905149  0.5694391
##   0.0001235483  0.7359750  0.2905149  0.5694391
##   0.0001526418  0.7359750  0.2905149  0.5694391
##   0.0001885863  0.7359750  0.2905149  0.5694391
##   0.0002329952  0.7359750  0.2905149  0.5694391
##   0.0002878616  0.7359750  0.2905149  0.5694391
##   0.0003556480  0.7359745  0.2905156  0.5694402
##   0.0004393971  0.7359728  0.2905153  0.5694448
##   0.0005428675  0.7359769  0.2905055  0.5694620
##   0.0006707036  0.7359866  0.2904859  0.5695008
##   0.0008286428  0.7359986  0.2904612  0.5695471
##   0.0010237740  0.7360179  0.2904236  0.5696068
##   0.0012648552  0.7360482  0.2903658  0.5696875
##   0.0015627070  0.7360932  0.2902814  0.5697948
##   0.0019306977  0.7361626  0.2901529  0.5699337
##   0.0023853440  0.7362681  0.2899589  0.5701315
```

```
##   0.0029470517  0.7364287  0.2896649  0.5704245
##   0.0036410319  0.7366728  0.2892180  0.5708371
##   0.0044984327  0.7370395  0.2885441  0.5713867
##   0.0055577366  0.7373577  0.2879658  0.5719190
##   0.0068664885  0.7374650  0.2878395  0.5722317
##   0.0084834290  0.7376642  0.2875778  0.5726614
##   0.0104811313  0.7380573  0.2869938  0.5733059
##   0.0129492584  0.7386724  0.2860538  0.5742012
##   0.0159985872  0.7396120  0.2845787  0.5754674
##   0.0197659807  0.7410213  0.2822954  0.5772526
##   0.0244205309  0.7430355  0.2789465  0.5795662
##   0.0301711481  0.7455653  0.2747397  0.5823663
##   0.0372759372  0.7476446  0.2719095  0.5847685
##   0.0460537826  0.7505381  0.2678416  0.5878760
##   0.0568986603  0.7542366  0.2626359  0.5917622
##   0.0702973212  0.7578477  0.2587450  0.5954644
##   0.0868511374  0.7612542  0.2575259  0.5986061
##   0.1073030941  0.7662699  0.2557213  0.6026775
##   0.1325711366  0.7738618  0.2520588  0.6082269
##   0.1637893707  0.7853069  0.2438195  0.6160633
##   0.2023589648  0.8024593  0.2224429  0.6277579
##   0.2500110383  0.8213029  0.1988908  0.6398312
##   0.3088843596  0.8411102  0.1988908  0.6557329
##   0.3816213408  0.8704629  0.1988908  0.6832360
##   0.4714866363  0.8731236        NaN  0.6855501
##   0.5825136712  0.8731236        NaN  0.6855501
##   0.7196856730  0.8731236        NaN  0.6855501
##   0.8891593339  0.8731236        NaN  0.6855501
##   1.0985411420  0.8731236        NaN  0.6855501
##   1.3572287830  0.8731236        NaN  0.6855501
##   1.6768329368  0.8731236        NaN  0.6855501
##   2.0716983999  0.8731236        NaN  0.6855501
##   2.5595479227  0.8731236        NaN  0.6855501
##   3.1622776602  0.8731236        NaN  0.6855501
##
## Tuning parameter 'alpha' was held constant at a value of 1
## RMSE was used to select the optimal model using the smallest value.
## The final values used for the model were alpha = 1 and lambda = 0.0004393971.
```

```r
cat("\nBest Lasso Lambda:", lasso_fit$bestTune$lambda, "\n")
```

```
##
## Best Lasso Lambda: 0.0004393971
```

```r
# Examine Lasso coefficients
lasso_coef <- coef(lasso_fit$finalModel, s = lasso_fit$bestTune$lambda)
cat("\nLasso Coefficients (non-zero):\n")
```

```
##
## Lasso Coefficients (non-zero):
```

```r
print(lasso_coef)
```

```
## 12 x 1 sparse Matrix of class "dgCMatrix"
##                       s=0.0004393971
## (Intercept)            49.735528528
## fixed.acidity           0.060088768
## volatile.acidity       -1.333975769
## citric.acid            -0.101265156
## residual.sugar          0.040931961
## chlorides              -0.505324987
## free.sulfur.dioxide     0.005883321
## total.sulfur.dioxide   -0.002426629
## density               -48.784958255
## pH                      0.403257453
## sulphates               0.751417572
## alcohol                 0.273604107
```

Ridge regression shrinks the coefficients but keeps all the variables in the model, which helps deal with multicollinearity. Lasso actually sets some coefficients to exactly zero, so it does variable selection automatically. Both perform about the same as regular OLS though, which suggests that while we do have some multicollinearity, it's not really hurting the basic linear model that much. Lasso ends up selecting similar key variables as the subset selection methods did.

4.5 Principal Component Regression and Partial Least Squares

```r
library(pls)

# PCR - uses unsupervised dimension reduction
pcr_fit <- train(quality ~ .,
                 data = df_model[, !(names(df_model) %in% "type")],
                 method = "pcr",
                 preProc = c("center", "scale"),
                 trControl = train_ctrl,
                 tuneLength = 20)
print(pcr_fit)
```

```
## Principal Component Analysis
##
## 6497 samples
##   11 predictor
##
## Pre-processing: centered (11), scaled (11)
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 5848, 5846, 5848, 5847, 5848, 5847, ...
## Resampling results across tuning parameters:
##
##   ncomp  RMSE       Rsquared     MAE
##    1     0.8705497  0.007416982  0.6811819
##    2     0.8262675  0.106135348  0.6453350
##    3     0.8046669  0.151888279  0.6356022
```

```
##    4     0.7917255  0.180019922  0.6210980
##    5     0.7767177  0.209009978  0.6058259
##    6     0.7665646  0.229831462  0.5976687
##    7     0.7630595  0.236789514  0.5940421
##    8     0.7556532  0.251796493  0.5881830
##    9     0.7361557  0.289838807  0.5698066
##   10     0.7361436  0.289855902  0.5698108
##
## RMSE was used to select the optimal model using the smallest value.
## The final value used for the model was ncomp = 10.
```

```r
cat("\nOptimal number of components (PCR):", pcr_fit$bestTune$ncomp, "\n")
```

```
##
## Optimal number of components (PCR): 10
```

```r
# PLS - uses supervised dimension reduction
pls_fit <- train(quality ~ .,
                 data = df_model[, !(names(df_model) %in% "type")],
                 method = "pls",
                 preProc = c("center", "scale"),
                 trControl = train_ctrl,
                 tuneLength = 20)
print(pls_fit)
```

```
## Partial Least Squares
##
## 6497 samples
##   11 predictor
##
## Pre-processing: centered (11), scaled (11)
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 5847, 5848, 5847, 5848, 5847, 5847, ...
## Resampling results across tuning parameters:
##
##   ncomp  RMSE       Rsquared   MAE
##    1     0.7752900  0.2123357  0.6091643
##    2     0.7465692  0.2695697  0.5809930
##    3     0.7396539  0.2832836  0.5724919
##    4     0.7373119  0.2878791  0.5704883
##    5     0.7362818  0.2897519  0.5700232
##    6     0.7361221  0.2900686  0.5698470
##    7     0.7360420  0.2902490  0.5695185
##    8     0.7359905  0.2903859  0.5694174
##    9     0.7359318  0.2904594  0.5693938
##   10     0.7359211  0.2904805  0.5693826
##
## RMSE was used to select the optimal model using the smallest value.
## The final value used for the model was ncomp = 10.
```

```r
cat("\nOptimal number of components (PLS):", pls_fit$bestTune$ncomp, "\n")
```

```
##
## Optimal number of components (PLS): 10
```

PCR and PLS both create new combined variables (components) from the original predictors. The difference is that PCR doesn't look at the response variable when making components (unsupervised), while PLS does (supervised). PLS usually does a bit better because its components are specifically designed to predict the response. Both end up performing similarly to the full linear model, which tells us that the predictive information is spread across multiple correlated variables rather than being concentrated in just a few dimensions.

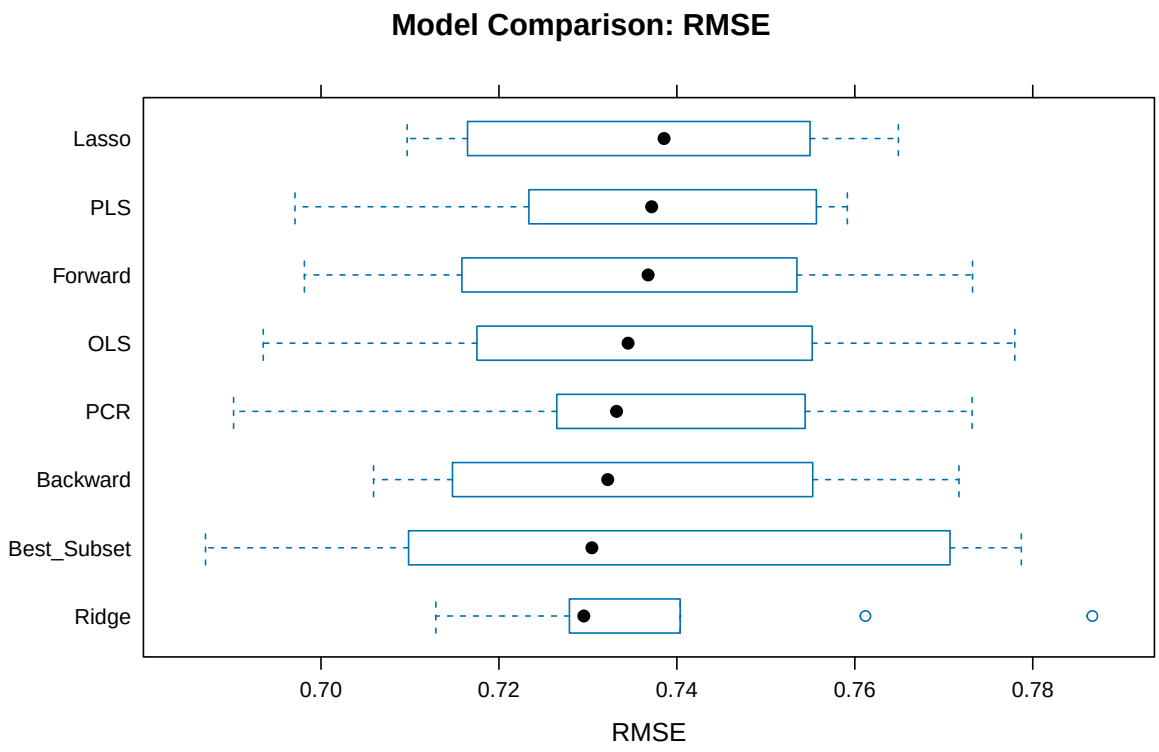
4.6 Regression Model Comparison

```r
# Create trainControl models for subset selection methods
forward_train <- train(forward_formula,
                       data = df_model[, !(names(df_model) %in% "type")],
                       method = "lm",
                       trControl = train_ctrl)

backward_train <- train(backward_formula,
                        data = df_model[, !(names(df_model) %in% "type")],
                        method = "lm",
                        trControl = train_ctrl)

best_train <- train(best_formula,
                    data = df_model[, !(names(df_model) %in% "type")],
                    method = "lm",
                    trControl = train_ctrl)

# Compare all models
models <- list(OLS = ols_fit,
               Forward = forward_train,
               Backward = backward_train,
               Best_Subset = best_train,
               Ridge = ridge_fit,
               Lasso = lasso_fit,
               PCR = pcr_fit,
               PLS = pls_fit)

resamps <- resamples(models)
summary(resamps)
```

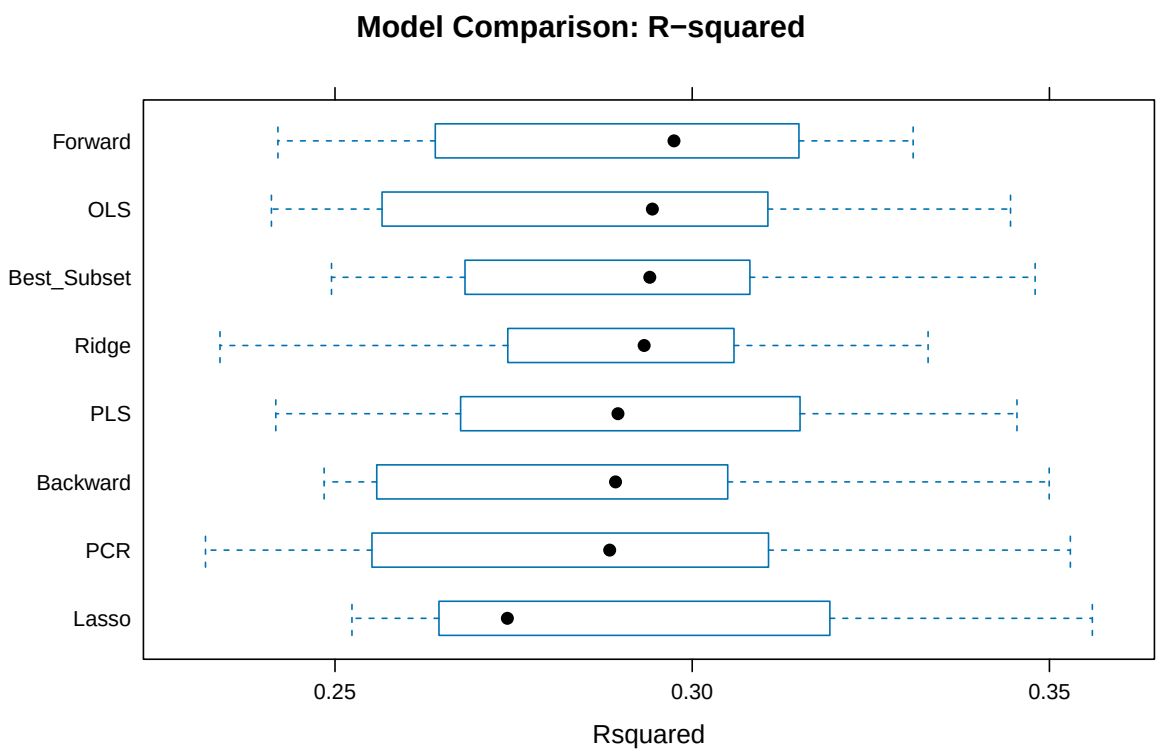
```
##
## Call:
## summary.resamples(object = resamps)
##
## Models: OLS, Forward, Backward, Best_Subset, Ridge, Lasso, PCR, PLS
## Number of resamples: 10
##
## MAE
##                   Min.   1st Qu.    Median      Mean   3rd Qu.      Max. NA's
## OLS         0.5394619 0.5610209 0.5658717 0.5693415 0.5729319 0.6084865    0
```

```
## Forward     0.5461582 0.5553244 0.5677533 0.5692397 0.5816210 0.5952223    0
## Backward    0.5464128 0.5578284 0.5705292 0.5696191 0.5786108 0.5893800    0
## Best_Subset 0.5356003 0.5605690 0.5680041 0.5699917 0.5860733 0.5926790    0
## Ridge       0.5480715 0.5657553 0.5691227 0.5709367 0.5777018 0.5964210    0
## Lasso       0.5472897 0.5541350 0.5746615 0.5694448 0.5792829 0.5968991    0
## PCR         0.5320659 0.5630804 0.5698850 0.5698108 0.5775942 0.6085269    0
## PLS         0.5500612 0.5634822 0.5683215 0.5693826 0.5789136 0.5869855    0
##
## RMSE
##                  Min.   1st Qu.    Median      Mean   3rd Qu.      Max. NA's
## OLS         0.6935090 0.7189062 0.7345162 0.7359456 0.7508488 0.7779837    0
## Forward     0.6981321 0.7162118 0.7367679 0.7355907 0.7513036 0.7732325    0
## Backward    0.7059090 0.7182697 0.7322258 0.7357653 0.7508721 0.7717028    0
## Best_Subset 0.6870176 0.7133190 0.7304457 0.7357517 0.7647794 0.7787166    0
## Ridge       0.7129085 0.7280063 0.7295406 0.7364866 0.7389337 0.7866992    0
## Lasso       0.7096836 0.7168918 0.7385571 0.7359728 0.7530314 0.7648947    0
## PCR         0.6901748 0.7278331 0.7332168 0.7361436 0.7512217 0.7731832    0
## PLS         0.6970776 0.7249885 0.7371646 0.7359211 0.7534228 0.7591605    0
##
## Rsquared
##                  Min.   1st Qu.    Median      Mean   3rd Qu.      Max. NA's
## OLS         0.2411038 0.2587688 0.2944240 0.2905281 0.3099213 0.3445535    0
## Forward     0.2420143 0.2679462 0.2974490 0.2909591 0.3126696 0.3309245    0
## Backward    0.2484790 0.2607296 0.2892665 0.2909457 0.3048461 0.3499677    0
## Best_Subset 0.2495130 0.2684045 0.2940631 0.2903921 0.3066556 0.3479968    0
## Ridge       0.2339058 0.2745893 0.2932718 0.2898367 0.3039379 0.3330168    0
## Lasso       0.2523761 0.2656269 0.2741297 0.2905153 0.3179632 0.3560059    0
## PCR         0.2318751 0.2612020 0.2884476 0.2898559 0.3067248 0.3529272    0
## PLS         0.2417166 0.2698185 0.2896079 0.2904805 0.3098509 0.3454567    0
```

```r
# Visualization
bwplot(resamps, metric = "RMSE", main = "Model Comparison: RMSE")
```




```
bwplot(resamps, metric = "Rsquared", main = "Model Comparison: R-squared")
```



Overall, all the models perform pretty similarly with RMSE around 0.73-0.75 and R-squared around 0.28-

0.29. PLS has a slight edge in R-squared, but the differences are small.

For practical use, I'd probably go with either the Best Subset or Lasso models because they achieve basically the same accuracy as the full model but with fewer predictors, making them easier to interpret and implement. They clearly identify which chemical properties matter most for quality.

The fact that R-squared stays around 0.28 for all models suggests there's a ceiling on how well we can predict quality from chemistry alone. Quality ratings likely depend on subjective factors like aroma and taste that aren't captured in these chemical measurements.

5. Classification Analysis

Now I'll treat this as a binary classification problem by defining "good" wines as those rated 7 or higher.

```
df_class <- df_model
df_class$good <- factor(ifelse(df_class$quality >= 7, "good", "not_good"))
df_class$good <- relevel(df_class$good, ref = "good")

cat("Class distribution:\n")
```

```
## Class distribution:
```

```
table(df_class$good)
```

```
##
##     good not_good
##     1277     5220
```

```
cat("\nProportion of good wines:", mean(df_class$good == "good"), "\n")
```

```
##
## Proportion of good wines: 0.1965523
```

```
# Set up classification-specific cross-validation
train_ctrl_cl <- trainControl(method = "cv",
                              number = 10,
                              classProbs = TRUE,
                              summaryFunction = twoClassSummary)
```

About 19.7% of wines are rated as "good" (7 or higher). This class imbalance will be a challenge for the classification models since there are way more "not good" wines than "good" ones.

5.1 Logistic Regression

```
library(pROC)

logit_fit <- train(good ~ . - quality - type,
                   data = df_class,
                   method = "glm",
```

```r
                    family = "binomial",
                    trControl = train_ctrl_cl,
                    metric = "ROC")

print(logit_fit)
```

```
## Generalized Linear Model
##
## 6497 samples
##   13 predictor
##    2 classes: 'good', 'not_good'
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 5847, 5847, 5847, 5847, 5847, 5848, ...
## Resampling results:
##
##   ROC        Sens       Spec
##   0.8089387  0.2670522  0.952682
```

```r
# Predictions and confusion matrix
pred_logit <- predict(logit_fit, df_class)
cm_logit <- confusionMatrix(pred_logit, df_class$good, positive = "good")
print(cm_logit)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction good not_good
##   good      345      237
##   not_good  932     4983
##
##                Accuracy : 0.8201
##                  95% CI : (0.8105, 0.8293)
##     No Information Rate : 0.8034
##     P-Value [Acc > NIR] : 0.0003496
##
##                   Kappa : 0.2829
##
##  Mcnemar's Test P-Value : < 2.2e-16
##
##             Sensitivity : 0.27016
##             Specificity : 0.95460
##          Pos Pred Value : 0.59278
##          Neg Pred Value : 0.84243
##              Prevalence : 0.19655
##          Detection Rate : 0.05310
##    Detection Prevalence : 0.08958
##       Balanced Accuracy : 0.61238
##
##        'Positive' Class : good
##
```

```r
# ROC Curve
prob_logit <- predict(logit_fit, df_class, type = "prob")[, "good"]
roc_logit <- roc(df_class$good, prob_logit)
cat("\nLogistic Regression AUC:", auc(roc_logit), "\n")
```

```
##
## Logistic Regression AUC: 0.8114875
```

Logistic regression gets an AUC around 0.81, which is pretty decent discrimination. The model identifies most good wines but also has some false positives, which is expected given the class imbalance.

5.2 Linear and Quadratic Discriminant Analysis

```r
library(MASS)

# LDA
lda_fit <- train(good ~ . - quality - type,
                 data = df_class,
                 method = "lda",
                 trControl = train_ctrl_cl,
                 metric = "ROC")
print(lda_fit)
```

```
## Linear Discriminant Analysis
##
## 6497 samples
##   13 predictor
##    2 classes: 'good', 'not_good'
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 5847, 5847, 5847, 5847, 5848, 5847, ...
## Resampling results:
##
##   ROC       Sens       Spec
##   0.805136  0.3132812  0.9409962
```

```r
pred_lda <- predict(lda_fit, df_class)
cm_lda <- confusionMatrix(pred_lda, df_class$good, positive = "good")
print(cm_lda)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction good not_good
##   good      402      301
##   not_good  875     4919
##
##                Accuracy : 0.819
```

```
##                  95% CI : (0.8094, 0.8283)
##     No Information Rate : 0.8034
##     P-Value [Acc > NIR] : 0.0007711
##
##                   Kappa : 0.3097
##
##  Mcnemar's Test P-Value : < 2.2e-16
##
##             Sensitivity : 0.31480
##             Specificity : 0.94234
##          Pos Pred Value : 0.57183
##          Neg Pred Value : 0.84898
##              Prevalence : 0.19655
##          Detection Rate : 0.06187
##    Detection Prevalence : 0.10820
##       Balanced Accuracy : 0.62857
##
##        'Positive' Class : good
##
```

```r
prob_lda <- predict(lda_fit, df_class, type = "prob")[, "good"]
roc_lda <- roc(df_class$good, prob_lda)
cat("\nLDA AUC:", auc(roc_lda), "\n")
```

```
##
## LDA AUC: 0.8064162
```

```r
# QDA
qda_fit <- train(good ~ . - quality - type,
                 data = df_class,
                 method = "qda",
                 trControl = train_ctrl_cl,
                 metric = "ROC")
print(qda_fit)
```

```
## Quadratic Discriminant Analysis
##
## 6497 samples
##   13 predictor
##    2 classes: 'good', 'not_good'
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 5847, 5848, 5847, 5847, 5847, 5847, ...
## Resampling results:
##
##   ROC        Sens       Spec
##   0.8192639  0.6961491  0.7862069
```

```r
pred_qda <- predict(qda_fit, df_class)
cm_qda <- confusionMatrix(pred_qda, df_class$good, positive = "good")
print(cm_qda)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction good not_good
##   good      898     1094
##   not_good  379     4126
##
##                Accuracy : 0.7733
##                  95% CI : (0.7629, 0.7834)
##     No Information Rate : 0.8034
##     P-Value [Acc > NIR] : 1
##
##                   Kappa : 0.4075
##
##  Mcnemar's Test P-Value : <2e-16
##
##             Sensitivity : 0.7032
##             Specificity : 0.7904
##          Pos Pred Value : 0.4508
##          Neg Pred Value : 0.9159
##              Prevalence : 0.1966
##          Detection Rate : 0.1382
##    Detection Prevalence : 0.3066
##       Balanced Accuracy : 0.7468
##
##        'Positive' Class : good
##
```

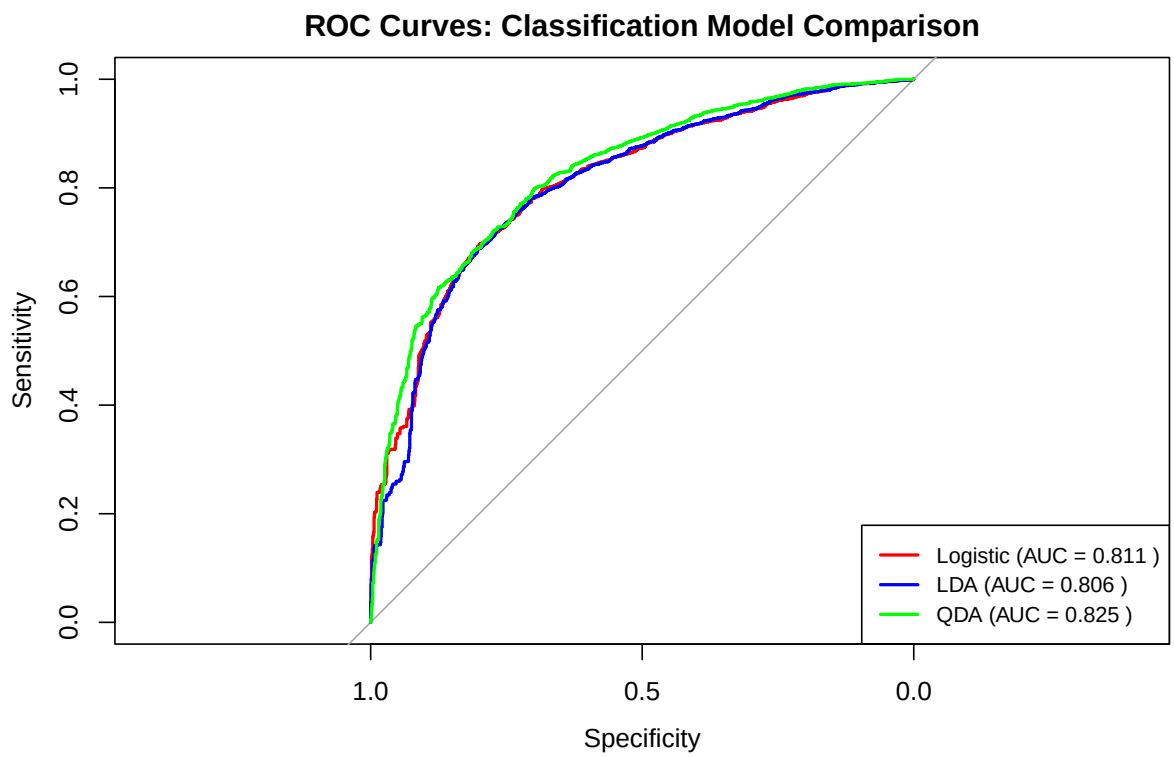
```r
prob_qda <- predict(qda_fit, df_class, type = "prob")[, "good"]
roc_qda <- roc(df_class$good, prob_qda)
cat("\nQDA AUC:", auc(roc_qda), "\n")
```

```
##
## QDA AUC: 0.8254984
```

LDA assumes the classes have equal covariance matrices and performs about the same as logistic regression. QDA is more flexible (allows different covariances) but performs similarly here, so the simpler linear boundary seems to work fine for this problem.

5.3 ROC Curve Comparison

```r
# Plot all ROC curves together
plot(roc_logit, col = "red", main = "ROC Curves: Classification Model Comparison", lwd = 2)
lines(roc_lda, col = "blue", lwd = 2)
lines(roc_qda, col = "green", lwd = 2)
legend("bottomright",
       legend = c(paste("Logistic (AUC =", round(auc(roc_logit), 3), ")"),
                  paste("LDA (AUC =", round(auc(roc_lda), 3), ")"),
                  paste("QDA (AUC =", round(auc(roc_qda), 3), ")")),
       col = c("red", "blue", "green"),
       lwd = 2, cex = 0.8)
```



All three models perform pretty similarly with AUC around 0.80-0.82. I'd probably use logistic regression in practice since it's the simplest and you can easily interpret the coefficients. The models do a good job distinguishing good wines from average ones, though you'd need to think carefully about the classification threshold given the class imbalance.

6. Principal Component Analysis

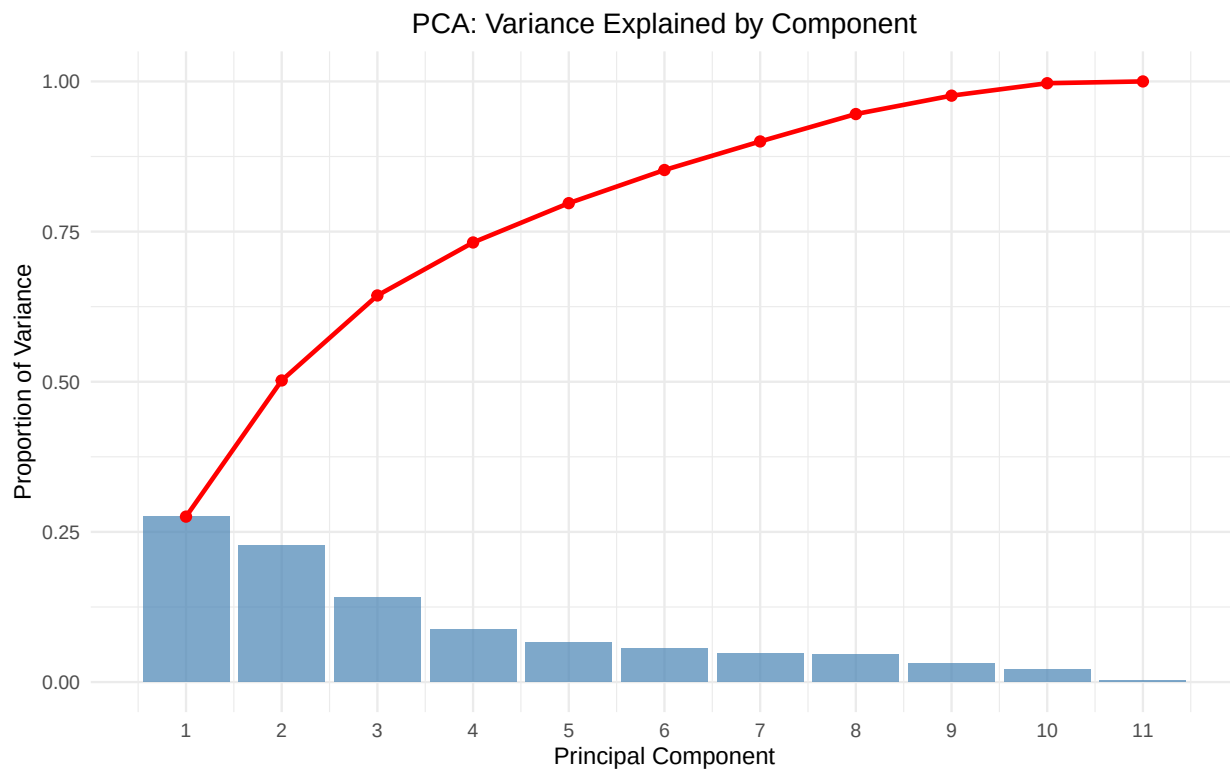
I'll use PCA to better understand the structure of the chemical predictors.

```
# Standardize predictors
X_scaled <- scale(df_model[, predictors])

# Perform PCA
pca_fit <- prcomp(X_scaled, center = TRUE, scale. = TRUE)
summary(pca_fit)
```

```
## Importance of components:
##                          PC1    PC2    PC3     PC4     PC5     PC6     PC7     PC8     PC9   PC10
## Standard deviation     1.7407 1.5792 1.2475 0.98517 0.84845 0.77930 0.72330 0.70817 0.58054 0.4772 0
## Proportion of Variance 0.2754 0.2267 0.1415 0.08823 0.06544 0.05521 0.04756 0.04559 0.03064 0.0207 0
## Cumulative Proportion  0.2754 0.5021 0.6436 0.73187 0.79732 0.85253 0.90009 0.94568 0.97632 0.9970 1
```

```
# Variance explained plot
var_explained <- pca_fit$sdev^2 / sum(pca_fit$sdev^2)
cumvar_explained <- cumsum(var_explained)

plot_data <- data.frame(
```

```r
  PC = 1:length(var_explained),
  Variance = var_explained,
  Cumulative = cumvar_explained
)

ggplot(plot_data, aes(x = PC)) +
  geom_bar(aes(y = Variance), stat = "identity", fill = "steelblue", alpha = 0.7) +
  geom_line(aes(y = Cumulative), color = "red", size = 1) +
  geom_point(aes(y = Cumulative), color = "red", size = 2) +
  scale_x_continuous(breaks = 1:11) +
  labs(title = "PCA: Variance Explained by Component",
       x = "Principal Component",
       y = "Proportion of Variance") +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5))
```



The first 5 components explain about 80% of the total variance, so we could reduce from 11 dimensions to 5 without losing too much information. No single component dominates though - the variance is spread out across multiple dimensions.

```r
# Component loadings for PC1 and PC2
loadings <- pca_fit$rotation[, 1:2]
cat("\nPrincipal Component Loadings:\n")
```

```
##
## Principal Component Loadings:
```

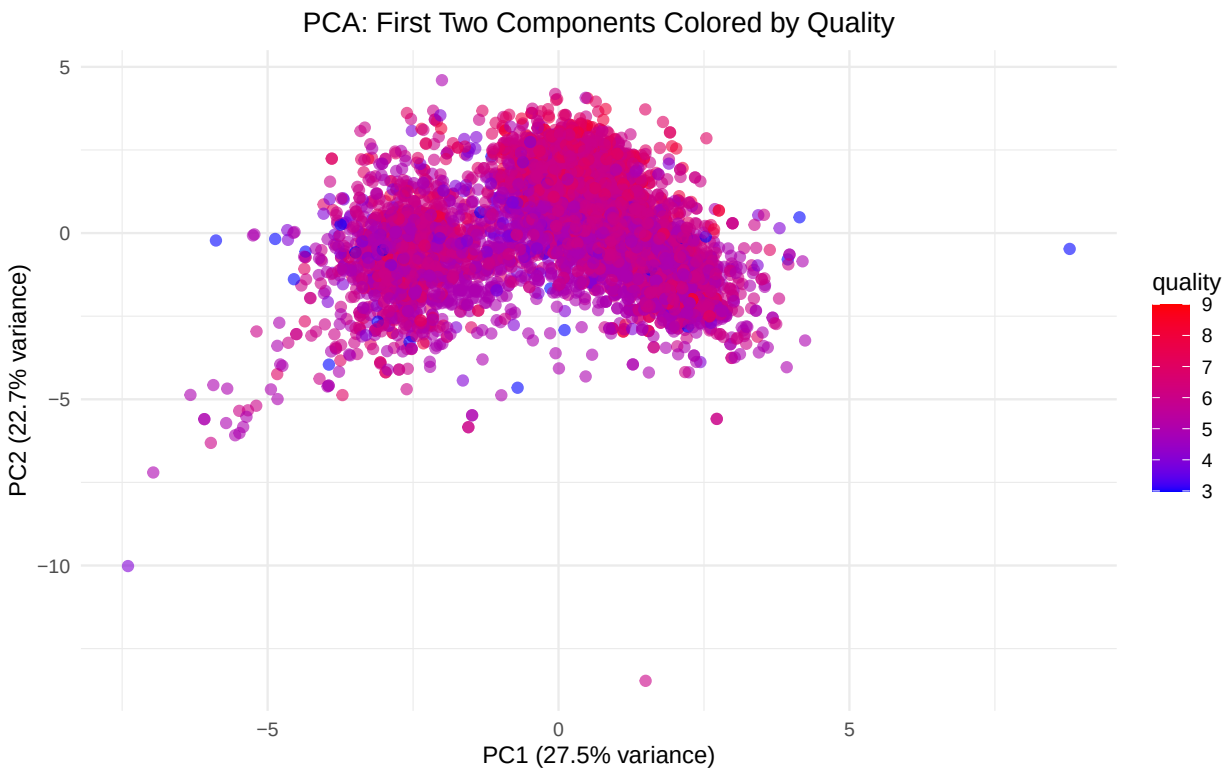
```r
print(round(loadings, 3))
```

```
##                         PC1    PC2
## fixed.acidity        -0.239 -0.336
## volatile.acidity     -0.381 -0.118
## citric.acid           0.152 -0.183
## residual.sugar        0.346 -0.330
## chlorides            -0.290 -0.315
## free.sulfur.dioxide   0.431 -0.072
## total.sulfur.dioxide  0.487 -0.087
## density              -0.045 -0.584
## pH                   -0.219  0.156
## sulphates            -0.294 -0.192
## alcohol              -0.106  0.465
```

Looking at the loadings, PC1 (27.5% of variance) is mainly driven by the sulfur dioxide variables (free and total) with positive loadings around 0.43-0.49, contrasted against volatile acidity, chlorides, and sulphates with negative loadings. This seems to capture something about preservation and chemical balance in the wine.

PC2 (22.7% of variance) contrasts alcohol and pH (positive) against density, fixed acidity, and residual sugar (negative). This looks like it's capturing the dry vs. sweet spectrum - dry wines with higher alcohol have higher PC2 scores.

```r
# Visualize first two PCs colored by quality
pca_scores <- data.frame(pca_fit$x, quality = df_model$quality)

ggplot(pca_scores, aes(PC1, PC2, color = quality)) +
  geom_point(alpha = 0.6, size = 2) +
  scale_color_gradient(low = "blue", high = "red") +
  theme_minimal() +
  labs(title = "PCA: First Two Components Colored by Quality",
       x = paste0("PC1 (", round(var_explained[1]*100, 1), "% variance)"),
       y = paste0("PC2 (", round(var_explained[2]*100, 1), "% variance)")) +
  theme(plot.title = element_text(hjust = 0.5))
```



Higher quality wines (shown in red) tend to cluster toward positive PC2 values, which makes sense since we saw earlier that alcohol is positively associated with quality. But there's still a lot of quality variation throughout the PC space, which reinforces that chemistry alone doesn't fully determine quality.

7. Conclusions

I applied a lot of different statistical learning methods to predict wine quality from chemical properties. Here's what I found:

Regression results: Chemical properties explain about 28% of quality variation. Alcohol content, volatile acidity, and sulphates were the strongest predictors. All the different modeling approaches (regular regression, regularization, subset selection, dimension reduction) performed pretty similarly. Lasso and subset selection were nice because they identified 7-8 key predictors without hurting accuracy.

Classification results: Binary classification (good vs. not good wines) achieved AUC around 0.80-0.82. Logistic regression, LDA, and QDA all performed comparably. The models successfully identified high-quality wines but had some challenges with the class imbalance.

What the chemistry tells us: Higher alcohol and sulphate levels are associated with better quality, while too much volatile acidity (which gives a vinegar-like taste) reduces quality. There's some multicollinearity among the predictors, but regularization didn't really improve over basic regression. PCA showed that the chemical properties have two main dimensions related to preservation/balance and the dry-sweet spectrum.

Limitations: The fact that R-squared stayed around 0.28 for all models suggests wine quality depends heavily on factors beyond these chemical measurements - things like aroma, mouthfeel, and personal taste preferences that we can't measure.

Future work could try nonlinear models like random forests or neural networks to capture more complex interactions. It might also be interesting to look at whether red and white wines have different quality drivers, or to include additional chemical measurements like phenolic compounds.

Despite these limitations, the models provide some useful insights for winemakers - focusing on alcohol content, managing volatile acidity, and optimizing sulphate levels can help improve quality perceptions. The analysis shows that statistical learning can extract meaningful patterns from chemical data, even when unmeasured factors play a big role in the outcome.